

COMPLETE VIVA GUIDE

[Full Explanation](#) · [Algorithms](#) · [Special Features](#) · [Dataset](#)

Python	FastAPI	NLTK	spaCy
TF-IDF	T5 Transformer	PyPDF2	SQLite
Named Entity Recognition	React Frontend	REST API	Flan-T5

TABLE OF CONTENTS

1. Project Introduction & Objective
2. Complete System Architecture
3. Dataset — What It Is & Where to Get It
4. Technology Stack — Every Library Explained
5. Algorithm 1 — TF-IDF (Extractive Summarization)
6. Algorithm 2 — T5 Transformer (Abstractive Summarization)
7. Algorithm 3 — NLP Preprocessing Pipeline
8. Algorithm 4 — Named Entity Recognition (spaCy)
9. Special Features of the Project
10.
 - . File-by-File Code Explanation
11.
 - . How the System Works — End to End
12.
 - . Frontend — React UI Explanation
13.
 - . Database Design
14.
 - . Viva Q&A; — 20 Questions with Detailed Answers

1. Project Introduction & Objective

The AI Government Policy Summarizer is a full-stack web application that takes any government policy PDF document and automatically converts it into a short, structured, citizen-friendly summary. It uses Natural Language Processing (NLP) and Artificial Intelligence (AI) algorithms entirely in Python to extract, clean, analyse, and summarise the document.

Problem it solves:

- Government policy documents are often 20–100 pages long, written in complex legal and bureaucratic language
- Ordinary citizens cannot understand what the policy means for them
- This project automatically extracts the key information and presents it in plain, simple language
- It saves hours of reading by producing a structured summary in under 2 minutes

What the system produces for each document:

Output	Description
Policy Summary	A 150–200 word AI-generated overview of the entire document
Key Points	5 most important sentences extracted from the policy
Keywords	Top 10 significant terms that define the document
Named Entities	All organisations, dates, budget amounts, laws, locations identified
Citizen Explanation	A plain-language version any person can understand

2. Complete System Architecture

The project follows a classic **Client-Server architecture** with a dedicated AI processing layer. All intelligence sits in the Python backend — the browser frontend is only for display.

Layer	Component	Language	Role
Presentation	React + HTML	JavaScript	User interface — upload, display results, history
API Gateway	FastAPI	Python	Receives HTTP requests, validates input, sends responses
Business Logic	pipeline.py	Python	Coordinates all AI modules in correct order
NLP Processing	preprocessing.py	Python	Cleans and prepares text for AI models
Summarization	summarizer.py	Python	TF-IDF scoring + T5 neural network summarization
Entity Analysis	keywords_ner.py	Python	Keyword frequency + spaCy NER model
Simplification	simplifier.py	Python	Flan-T5 instruction-tuned explanation generation
PDF Reading	pdf_extractor.py	Python	PyPDF2 binary parsing and text extraction
Data Storage	database.py	Python	SQLite file-based database, no server needed

Data flow: PDF file → PyPDF2 reads text → NLTK cleans text → TF-IDF selects key sentences → T5 rewrites into summary → spaCy finds entities → Flan-T5 simplifies → JSON returned to browser → React displays results

3. Dataset — What It Is & Where to Get It

This project does NOT use a fixed training dataset. It is an inference-only system — the AI models (T5, Flan-T5, spaCy) were pre-trained by Google and the spaCy team on massive datasets, and we use them directly. The "dataset" for this project is simply any government policy PDF the user uploads.

What kind of PDFs work best (your project dataset):

- Any government policy document in English with a digital text layer (not scanned)
- Minimum 2–3 pages for meaningful summarization
- Recommended: 5–30 pages for best results

Free sample PDF sources for demo/testing:

Source	URL	Type of Policy
India Policy Portal	indiacode.nic.in	Central Government Acts & Policies
PRS Legislative Research	prsindia.org	Bills, Acts, Committee Reports
Ministry of Education	education.gov.in	Education policies, NEP documents
World Bank Open Docs	documents.worldbank.org	Development policy reports
UK Government	gov.uk/government/publications	Clean, well-structured English PDFs
Your SIH Provided PDF	Provided by hackathon organizers	Government policy sample

Pre-trained model datasets (for viva reference):

Model	Pre-trained On	Size	Purpose in Project
google/flan-t5-base	C4 corpus (750GB text) + FLAN instruction data	250M parameters	Summarization & simplification
en_core_web_sm (spaCy)	OntoNotes 5.0 + Common Crawl	12MB	Named Entity Recognition
NLTK stopwords	Manually curated English word lists	—	Stopword removal
NLTK WordNet	Princeton WordNet lexical database	—	Lemmatization

4. Technology Stack — Every Library Explained

Library/Tool	Version	Category	Why This Specific Library
Python	3.9+	Core Language	Rich AI/ML ecosystem, clean syntax, all major NLP libraries support Python
FastAPI	0.111	Web Framework	Async-ready, auto-generates /docs API page, faster than Flask, built-in validation
uvicorn	0.29	ASGI Server	Runs FastAPI; ASGI supports async, handles concurrent requests better than WSGI
PyPDF2	3.0	PDF Extraction	Pure Python, no external dependencies, reads digital PDFs reliably
NLTK	3.8	NLP Preprocessing	Industry-standard NLP toolkit, includes tokenizers, stopwords, lemmatizer
spaCy	3.7	NER / NLP	Production-grade NLP, fastest NER in Python, en_core_web_sm model is small and fast
Transformers	4.40	AI Models	HuggingFace library — standard way to load T5/Flan-T5 models in Python
torch	2.3	Deep Learning	PyTorch backend required by Transformers; device=-1 forces CPU mode
sqlite3	Built-in	Database	No installation, file-based DB, perfect for single-user local projects
uuid	Built-in	Utilities	Generates collision-free unique IDs for each uploaded document
re	Built-in	Regex	Cleans PDF text artifacts using regular expressions
math	Built-in	Mathematics	log() function used in TF-IDF scoring formula
collections	Built-in	Data Structures	Counter class used for keyword frequency counting
React 18	CDN	Frontend	Component-based UI, handles state management for upload/results flow

5. Algorithm 1 — TF-IDF (Extractive Summarization)

TF-IDF (Term Frequency — Inverse Document Frequency) is a statistical algorithm used in information retrieval and text mining. In this project it is used to score every sentence in the document and identify the most important ones.

What is TF-IDF:

TF-IDF assigns a numerical score to each word in each sentence. Words that appear frequently in a specific sentence but rarely across the whole document get the highest score — they are the most distinctive and important words.

Formula:

$TF(\text{word}, \text{sentence}) = \text{count of word in sentence} / \text{total words in sentence}$

$IDF(\text{word}) = \log((N + 1) / (\text{document_frequency} + 1))$

$TF\text{-}IDF(\text{word}) = TF \times IDF$

$\text{Sentence Score} = \text{sum of TF-IDF scores of all words in the sentence}$

Step-by-step how it works in the code:

- Step 1: Split cleaned document text into individual sentences
- Step 2: For each word in each sentence, calculate TF — how common is this word in THIS sentence
- Step 3: For each word, calculate IDF — how rare is this word across ALL sentences
- Step 4: Multiply TF x IDF to get the importance score for each word in each sentence
- Step 5: Sum all word scores in a sentence to get the total sentence score
- Step 6: Add a policy-keyword boost — sentences containing words like "shall", "must", "fund", "ministry" get extra score
- Step 7: Select top 5–6 highest-scoring sentences
- Step 8: Re-sort selected sentences in their original document order (preserves logical flow)

Worked example:

Sentence	Key Words Found	Score (approx)	Selected?
"Every district shall establish a Digital Learning Center by 2025"	district, shall, digital, learning, center (all rare & specific)	High (4.2)	YES
"The government is committed to this program."	government, committed, program (common words)	Lower (1.1)	NO
"■ 2400 crore allocated to fund 28 states under this scheme"	allocated, fund, states, scheme (specific policy terms)	High (3.9)	YES

Why TF-IDF over other methods:

- No model training required — purely mathematical, works on any document
- Fast — runs in milliseconds even on long documents
- Transparent — you can see exactly why each sentence was selected
- Language-agnostic — works for any English policy document without fine-tuning

6. Algorithm 2 — T5 Transformer (Abstractive Summarization)

T5 (Text-to-Text Transfer Transformer) is a deep learning neural network model developed by Google in 2019. It treats every NLP task as a text-to-text problem — the input is text and the output is text. For summarization, the input is the document and the output is the summary.

How Transformer architecture works (simplified):

- **ENCODER:** Reads the input text and creates a mathematical representation (called embeddings) that captures the meaning of each word in context of all other words
- **ATTENTION MECHANISM:** For each word being processed, the model calculates how much "attention" to pay to every other word. "funds" pays high attention to "crore", "ministry", "allocated" — understanding the financial context
- **DECODER:** Generates the output summary one word at a time, using the encoder representation and previously generated words to decide what comes next

What is Flan-T5 (used in simplifier):

Flan-T5 is T5 that has been additionally trained on over 1,800 NLP tasks with instructions. This "instruction tuning" means it can follow commands like "Explain in simple words for citizens." Regular T5 just summarizes; Flan-T5 follows the instruction and changes its output style accordingly.

Two-stage summarization strategy:

Stage	Input	Process	Output	Words
Stage 1 (TF-IDF)	Full document	Mathematical scoring	Top 6 sentences	~300 words
Stage 2 (T5)	TF-IDF output	Neural generation	Final summary	150–200 words

Why two stages? T5 has a maximum input limit (~512 tokens = ~380 words). A 50-page policy document can be 15,000+ words. Feeding it directly would truncate most of the content. TF-IDF first intelligently selects the most important 300 words, which T5 then reads fully and rewrites into a clean, fluent summary.

Key parameters used:

```
model("text", max_length=200, min_length=60, do_sample=False, truncation=True, device=-1)
```

- `max_length=200` — summary will not exceed 200 tokens (~150 words)
- `min_length=60` — forces the model to produce at least 60 tokens, avoiding very short outputs
- `do_sample=False` — uses greedy decoding, produces deterministic output (same input = same output every time)
- `truncation=True` — if input exceeds token limit, truncate rather than crash
- `device=-1` — force CPU mode, no GPU required

7. Algorithm 3 — NLP Preprocessing Pipeline (NLTK)

Before any AI model processes the text, it must be cleaned and standardised. Raw PDF text contains page numbers, extra spaces, special characters, and common words that would confuse the models. The preprocessing pipeline runs 5 operations in sequence.

Step	Operation	Library	Input → Output	Why needed
1	Text Cleaning	Python re (regex)	Raw PDF text → Clean text	Removes page numbers, extra spaces, non-printable characters
2	Sentence Tokenization	NLTK sent_tokenize	Clean text → List of sentences	Splits document into processable units for TF-IDF
3	Word Tokenization	NLTK word_tokenize	Each sentence → List of words	Breaks text into individual tokens for analysis
4	Stopword Removal	NLTK stopwords	All tokens → Meaningful tokens only	Removes "the", "is", "at", "and" — 179 words removed
5	Lemmatization	NLTK WordNetLemmatizer	Tokens → Root forms	"running" → "run", "policies" → "policy", reduces vocabulary

Tokenization explained:

Tokenization is the process of splitting text into smaller units called tokens. Sentence tokenization uses period, exclamation mark, and question mark as boundaries — but intelligently handles abbreviations like "Dr." or "Rs." that should not split the sentence. Word tokenization splits by spaces and punctuation, keeping words and separating punctuation marks.

Input: "The policy shall provide funds. Every district must comply."

Output: ["The policy shall provide funds.", "Every district must comply."]

Stopword removal — why 179 words are removed:

NLTK's English stopwords list contains 179 words including: the, is, are, was, were, be, been, a, an, and, or, but, in, on, at, to, for, of, with, by, from, it, its, this, that, these, those, we, our, they, their, which, what, who, how. These words appear in almost every sentence and carry no unique meaning. Removing them ensures TF-IDF gives high scores only to meaningful content words.

Lemmatization vs Stemming — why lemmatization is used:

Word	Stemming Result	Lemmatization Result	Which is correct?
running	runn	run	Lemmatization
policies	polici	policy	Lemmatization
better	better	good	Lemmatization
established	establish	establish	Both same
studies	studi	study	Lemmatization

Stemming just chops off suffixes — it produces non-words like "runn" and "polici". Lemmatization uses a vocabulary database (WordNet) to find the actual dictionary form. This project uses lemmatization because NLP models understand real words, not stems.

8. Algorithm 4 — Named Entity Recognition / spaCy

Named Entity Recognition (NER) is an NLP task that identifies and classifies named entities in text — proper nouns that refer to real-world objects like organisations, dates, amounts, and locations. spaCy's NER uses a **Convolutional Neural Network (CNN)** model trained on the OntoNotes 5.0 dataset.

How spaCy NER works:

- **TOKENIZATION:** Text is split into tokens (words and punctuation)
- **FEATURE EXTRACTION:** For each token, the model considers: the word itself, its part of speech, its position in the sentence, and the surrounding words
- **IOB TAGGING:** Each token gets tagged as B-TYPE (Beginning of entity), I-TYPE (Inside entity), or O (Outside/not an entity). Example: "Ministry/B-ORG of/I-ORG Education/I-ORG held/O a/O meeting/O"
- **ENTITY ASSEMBLY:** B and I tags are combined to form complete entities: "Ministry of Education" → ORG

Entity types detected in this project:

Label	Full Name	Examples from policy docs	Badge Color
ORG	Organisation	Ministry of Electronics, NASSCOM, ISRO, State Govt	Green
GPE	Geo-Political Entity	India, Maharashtra, New Delhi, Mumbai	Blue
LOC	Location	Himalayan region, coastal areas, rural districts	Blue
DATE	Date/Time	March 2025, financial year 2024–25, Q3 2023	Yellow
MONEY	Monetary Amount	₹2400 crore, USD 500 million, Rs. 10,000	Amber
LAW	Law or Policy	Right to Education Act, NEP 2020, IT Act 2000	Purple
PERSON	Person Name	Prime Minister, Secretary General, Dr. Rajan	Pink

Why spaCy over other NER tools:

- **Speed:** spaCy is 10-40x faster than NLTK's NER at runtime
- **Accuracy:** en_core_web_sm achieves ~85% F1 score on standard benchmarks
- **Memory:** en_core_web_sm model is only 12MB — loads in under 2 seconds
- **Production-ready:** Used in real-world industry applications, not just research

9. Special Features of the Project

✓ Two-Stage AI Summarization

Most basic summarizers use only one method. This project chains TF-IDF (extractive) into T5 (abstractive) — first mathematically identifying the most important content, then using a neural network to rewrite it into fluent prose. This produces more accurate and readable summaries than either method alone.

✓ Citizen-Friendly Explanation Layer

Beyond just summarizing, the system has a dedicated simplification step using Flan-T5 with instruction prompting. The output is specifically designed for ordinary citizens without legal knowledge — avoiding jargon and using short, clear sentences. A rule-based fallback dictionary replaces complex legal terms if the model underperforms.

✓ CPU-Only Execution (No GPU Required)

All AI models run with `device=-1`, forcing CPU mode. The project is designed to run on any normal laptop without needing a dedicated graphics card (GPU). This is a deliberate design choice making the system accessible to anyone.

✓ Single-Load Model Architecture

All AI models (T5, Flan-T5, spaCy) are loaded once when the server starts and kept in memory for all subsequent requests. This prevents the 30–60 second model loading time from affecting every document processed. First load is slow; all subsequent requests are fast.

✓ REST API Architecture

The backend is a fully-featured REST API with three endpoints. This means the frontend and backend are completely decoupled — they communicate only through standard HTTP requests. The same backend could power a mobile app, a desktop application, or any other frontend without any changes.

✓ SQLite Persistence with History

Every processed document is saved to a SQLite database. Users can revisit any previous document without reprocessing. The History page shows all past analyses with metadata. Results are stored as JSON inside SQLite — a practical solution for array data in a relational database.

✓ Hero Landing Page with Scroll UX

The frontend uses a two-section scroll design. The first screen (100vh) shows the site name and tagline with an animated scroll arrow. The upload section is revealed on scrolling — a modern SaaS-style UX pattern that separates introduction from action.

✓ Graceful Error Handling

Every step has error handling: scanned PDFs return a clear message, network failures fall back to demo mode, model failures fall back to rule-based alternatives. The system never crashes silently — users always receive meaningful feedback.

10. File-by-File Code Explanation

backend/main.py

Python | FastAPI, uvicorn, uuid

The entry point and web server. Defines 3 API endpoints: /upload-policy (saves PDF to disk with UUID filename), /summarize-policy (runs full AI pipeline and returns JSON), /history (fetches all past results from database). Uses `@app.on_event("startup")` to load all AI models once at launch. `HTTPException` returns meaningful error messages for bad requests.

Key code points:

- `uuid.uuid4()` — generates a random unique ID for each PDF file
- `UploadFile = File(...)` — FastAPI reads the uploaded binary PDF
- `async def` — FastAPI supports async for better performance
- `JSONResponse` — returns Python dict as properly formatted JSON
- `Query(...)` — reads `document_id` from the URL query string

backend/database.py

Python | sqlite3 (built-in)

Manages the SQLite database file at `data/policy.db`. Creates the documents table on first run. Saves all pipeline results after each document is processed. Fetches document history sorted by creation time. Lists and arrays (`key_points`, `keywords`, `entities`) are stored as JSON strings because SQLite does not support array columns natively.

Key code points:

- `Path(__file__).parent.parent` — navigates up two folders from `backend/` to project root
- `DB.parent.mkdir(parents=True, exist_ok=True)` — auto-creates `data/` folder
- `json.dumps()` / `json.loads()` — serialize Python lists to/from JSON strings
- `INSERT OR REPLACE` — updates record if ID already exists
- `c.row_factory = sqlite3.Row` — returns rows as dictionaries, not tuples

modules/pdf_extractor.py

Python | PyPDF2

Opens the uploaded PDF in binary read mode (`"rb"`). Creates a `PdfReader` object and iterates through every page. Calls `page.extract_text()` on each page and joins all text. Returns `raw_text`, `page_count`, and `word_count` as a dictionary. Falls back to `PyMuPDF` (fitz) if `PyPDF2` is not installed.

Key code points:

- `open(path, "rb")` — PDFs are binary files, must be read in binary mode
- `PdfReader(f).pages` — list of all Page objects in the PDF
- `page.extract_text()` — extracts digital text layer from each page
- Raises `ValueError` if text is empty — scanned PDFs have no text layer

modules/preprocessing.py

Python | NLTK

Runs the 3-step NLP preprocessing pipeline on raw PDF text. Step 1: `clean_text()` uses regex to remove page numbers, extra whitespace, and non-printable characters. Step 2: `preprocess()` tokenizes into sentences and words. Step 3: Removes 179 English stopwords and lemmatizes remaining tokens.

Key code points:

- `re.sub(r"[^\x20-\x7E\n]", "", text)` — removes non-ASCII characters
- `re.sub(r"^\s*d+\s*$", "", text, MULTILINE)` — removes standalone page numbers
- `nltk.download(quiet=True)` — downloads NLTK data silently on first run
- `set(stopwords.words("english"))` — O(1) lookup, faster than list
- `LEM.lemmatize(token)` — uses WordNet to find correct base form

modules/summarizer.py

Python | math, collections, HuggingFace Transformers

Contains three functions: `extractive_summary()` uses TF-IDF to select the top sentences. `abstractive_summary()` feeds those sentences to T5 which rewrites them into fluent prose. `get_key_points()` selects the top 5 sentences from the full document (not the TF-IDF subset) as human-readable bullet points.

Key code points:

- `math.log()` — the IDF formula requires log to compress the scale of large document frequencies
- `Counter(words)` — counts word frequencies efficiently
- `do_sample=False` — greedy decoding, deterministic output
- `device=-1` — CPU inference, no GPU needed
- `Global_t5 = None` — model only loads once, singleton pattern

modules/keywords_ner.py

Python | collections.Counter, spaCy

Two functions: `extract_keywords()` finds top 10 most frequent non-stopword words using Counter. `extract_entities()` loads spaCy's `en_core_web_sm` model and runs NER on the first 6000 characters of text. Entities are deduplicated by `(text.lower(), label)` key to avoid repetition.

Key code points:

- `text[:6000]` — limit NER to first 6000 chars for speed (main entities appear early)
- `set()` for seen — O(1) deduplication lookup
- `doc.ents` — spaCy's list of detected named entity spans
- `ent.label_` — the entity type string: "ORG", "DATE", etc.
- `LABEL_MAP` dict — converts spaCy internal labels to human-readable descriptions

modules/simplifier.py

Python | HuggingFace Transformers (Flan-T5)

Uses Flan-T5 (instruction-tuned T5) with a carefully crafted prompt to convert the technical summary into plain citizen language. The prompt explicitly instructs the model to use short sentences and avoid jargon. If the model output is less than 15 words, it falls back to a rule-based dictionary replacement of complex legal terms.

Key code points:

- `text2text-generation` — Flan-T5 is used in generation mode, not just classification
- `max_new_tokens=200` — controls output length without counting input tokens

- Prompt engineering — prefixing with instruction changes model output style
 - Fallback dict — 7 legal-to-plain replacements guarantee some output always
-

modules/pipeline.py

Python | Imports all other modules

The orchestrator. PolicyPipeline class loads spaCy on `__init__`. `process(pdf_path)` calls all modules in sequence and returns one unified result dictionary. The `main.py` backend only needs to call `pipeline.process()` — it does not need to know about any individual module.

Key code points:

- Class pattern — encapsulates state (loaded models) and behavior (process method)
 - Single responsibility — each module does one thing, pipeline just coordinates
 - dict return — all results in one JSON-serializable dictionary
 - `print()` logs — terminal shows which step is running, useful for debugging
-

11. How the System Works — End to End

Step	What Happens	Time Taken	Module
1	User opens browser, sees hero page and uploads PDF	Instant	Frontend
2	React sends POST /upload-policy with PDF binary data	< 0.1 sec	main.py
3	FastAPI saves PDF as {uuid}.pdf in data/uploads/	< 0.1 sec	main.py
4	React sends POST /summarize-policy?document_id={id} & blocks until done	Blocks until done	main.py
5	PyPDF2 reads every page and extracts raw text string	~3 sec	pdf_extractor.py
6	NLTK cleans text, tokenizes, removes stopwords, lemmatizes	0.5–1 sec	preprocessing.py
7	TF-IDF scores all sentences, selects top 6	0.5–1 sec	summarizer.py
8	T5 model reads TF-IDF output, generates abstractive summary on CPU	20–60 sec	summarizer.py
9	TF-IDF runs again on full doc to select 5 key points	0.5 sec	summarizer.py
10	Counter finds top 10 most frequent meaningful words	< 0.1 sec	keywords_ner.py
11	spaCy NER model identifies all named entities in first 6000 chars	1–6 sec	keywords_ner.py
12	Flan-T5 with instruction prompt generates citizen explanation on CPU	10–30 sec	simplifier.py
13	All results saved to SQLite database	< 0.1 sec	database.py
14	JSON response returned to browser with all results	< 0.1 sec	main.py
15	React renders Summary, Key Points, Keywords, Entities, Explanation	Instant	Frontend

Total processing time on a normal laptop (CPU): approximately 45–90 seconds depending on document length. A 5-page PDF takes ~45 seconds. A 30-page PDF takes ~90 seconds. The bottleneck is always the T5 and Flan-T5 model inference on CPU.

12. Frontend — React UI Explanation

The frontend is a single HTML file that uses React 18 loaded from CDN. It has no build step — just open index.html in a browser. React manages UI state and makes HTTP calls to the Python backend.

Pages / Screens:

Screen	Trigger	What user sees
Hero Landing	Page load	Full-screen navy gradient, site name, tagline quote, bouncing scroll arrow
Upload Section	Scroll down	Drag-and-drop PDF box, Analyse button, demo button, info notice
Processing	Click Analyse	Step-by-step progress list, animated progress bar, filename shown
Results	Pipeline complete	Stats strip, Summary, Key Points, Keywords, Entities, Citizen Brief
History	Click History nav	List of all previously analysed documents with previews

Key React concepts used:

- `useState` — stores current page, result data, progress percentage, step label, filename
- `useEffect` — runs side effects: animating progress bar when progress state changes
- `useRef` — stores `setTimeout` reference so animation can be cancelled on reset
- `useCallback` — memoises the toast notification add function
- `fetch()` — native browser API used to call FastAPI endpoints asynchronously
- `async/await` — waits for API responses without blocking the UI thread

Colour scheme (matching your SIH slide):

Variable	Hex Value	Used For
--navy	#1a3a5c	Header background, section titles, primary text
--blue	#2563a8	Primary buttons, links
--blue-mid	#3b7cc9	Key point numbers, block headers, progress bar
--blue-pale	#dbeaf6	Block backgrounds, table alternating rows
--blue-bg	#eef5fb	Upload hover state, active step highlight

13. Database Design

The database is a single SQLite file at `data/policy.db`. It contains one table: **documents**. SQLite is a serverless database engine — the entire database is stored in one file on disk, managed entirely by Python's built-in `sqlite3` module.

Table: documents

Column	Data Type	Constraint	Description
id	TEXT	PRIMARY KEY	UUID string — unique identifier for each document
filename	TEXT	NOT NULL	Original name of the uploaded PDF file
summary	TEXT	—	Full AI-generated summary paragraph from T5
key_points	TEXT	—	JSON string: Python list of 5 key sentences
keywords	TEXT	—	JSON string: Python list of 10 keywords
entities	TEXT	—	JSON string: list of {text, label, description} dicts
citizen_explanation	TEXT	—	Plain language explanation from Flan-T5
word_count	INTEGER	—	Total number of words in original PDF
processing_time	REAL	—	Seconds taken by the full pipeline
created_at	TEXT	—	ISO 8601 timestamp: 2025-03-24T14:32:11.123456

How to view the database:

- VS Code: Install "SQLite Viewer" extension → click `data/policy.db`
- DB Browser for SQLite: Free app from sqlitebrowser.org → Open Database → select `policy.db`
- Command line: `sqlite3 data/policy.db` → then type: `SELECT * FROM documents;`

14. Viva Q&A; — 20 Questions with Detailed Answers

Q1: What is the main objective of your project?

A: The project converts complex government policy PDFs into structured, citizen-friendly summaries using AI and NLP. It solves the real problem that most citizens cannot understand the legal language of government documents. The system automatically extracts key information, identifies important terms, detects named entities, and rewrites everything in plain language — all within 2 minutes.

Q2: Which programming language is used and why?

A: Python is the main language for the entire backend, AI pipeline, and database. Python was chosen because it has the richest ecosystem of AI and NLP libraries: HuggingFace Transformers, NLTK, spaCy, FastAPI, and sqlite3 are all Python packages. The frontend uses HTML and JavaScript only for displaying results — it contains no AI logic.

Q3: What is TF-IDF? Explain the formula.

A: TF-IDF stands for Term Frequency — Inverse Document Frequency. It is a statistical algorithm that scores words based on how important they are in a specific sentence relative to the whole document. $TF = \text{count of word in sentence} / \text{total words in sentence}$. $IDF = \log((N+1)/(\text{document_frequency}+1))$. $TF-IDF = TF \times IDF$. Sentences with high-scoring words are selected as the most important sentences in the document.

Q4: What is the difference between extractive and abstractive summarization?

A: Extractive summarization selects and returns existing sentences from the original document based on their TF-IDF scores — no new text is generated. Abstractive summarization uses T5, a neural network that reads the content and generates entirely new sentences — like a human paraphrasing. This project uses both: TF-IDF first reduces the document to 300 words, then T5 rewrites it into a fluent 150-200 word summary.

Q5: What is the T5 model? Why is it used?

A: T5 stands for Text-to-Text Transfer Transformer. It is a deep learning model developed by Google that treats every NLP task as converting input text to output text. It uses an encoder-decoder Transformer architecture with an attention mechanism. It was pre-trained on 750GB of text (C4 corpus). We use google/flan-t5-base because it is instruction-tuned — it can follow commands like "explain in simple words" and it fits in RAM on a normal laptop (250M parameters).

Q6: What is Named Entity Recognition? What entities does your project detect?

A: Named Entity Recognition (NER) is an NLP task that identifies and classifies named entities — proper nouns referring to real-world objects. spaCy's NER model uses a CNN with IOB tagging to label each token. This project detects: ORG (organisations), GPE (locations/governments), DATE (dates), MONEY (budget amounts), LAW (laws/policies), PERSON (named people), and LOC (locations).

Q7: What is tokenization and why is it needed?

A: Tokenization is the process of splitting text into smaller units called tokens. Sentence tokenization splits by sentence boundaries. Word tokenization splits by words and punctuation. It is needed because AI models cannot directly process raw strings — they need structured lists of tokens to perform analysis, calculate frequencies, and assign scores.

Q8: What are stopwords and why are they removed?

A: Stopwords are extremely common words like "the", "is", "at", "and", "of" that appear in almost every sentence but carry no meaningful information. NLTK provides 179 English stopwords. Removing them ensures TF-IDF scoring focuses on meaningful content words. Without removal, "the" and "is" would always score highest because they appear most frequently — giving no useful information.

Q9: What is lemmatization and how is it different from stemming?

A: Lemmatization reduces words to their correct dictionary root form using the WordNet vocabulary database. "running" → "run", "policies" → "policy". Stemming just chops off suffixes: "policies" → "polici", "running" → "runn" — these are not real words. Lemmatization is preferred because it produces actual dictionary words that NLP models can correctly interpret.

Q10: What is FastAPI and why not Flask?

A: FastAPI is a modern Python web framework that builds REST APIs. It is faster than Flask, natively supports async/await, automatically generates interactive API documentation at /docs, and uses Python type hints for automatic request validation. When the user uploads a non-PDF file, FastAPI rejects it immediately with a proper error message before the Python code even runs.

Q11: Why is SQLite used instead of MySQL or PostgreSQL?

A: SQLite requires zero installation and zero configuration. The entire database is stored as one file (policy.db) in the project folder. Python's sqlite3 module is built-in — no extra package needed. For a single-user local project, SQLite is more than sufficient. MySQL and PostgreSQL require a separate running server, which adds unnecessary complexity for a demonstration project.

Q12: What does device=-1 mean in your code?

A: In HuggingFace Transformers, device parameter controls which hardware runs inference. device=-1 means CPU (central processor). device=0 means the first GPU. This project uses device=-1 so it runs on any normal laptop without needing a graphics card. The tradeoff is speed — CPU inference takes 30-90 seconds vs 2-5 seconds on GPU.

Q13: What happens if the PDF is scanned (image-only)?

A: PyPDF2 can only extract text if the PDF has a digital text layer. A scanned PDF is essentially a photograph — it contains only pixels, not text characters. extract_text_from_pdf() returns an empty string. The pipeline detects this, raises a ValueError with message "No text could be extracted. The PDF may be image-only." main.py catches it and returns HTTP 500 with the clear message to the user.

Q14: Explain the two-stage summarization strategy.

A: Stage 1: TF-IDF extractive summarization selects the top 6 most important sentences from the full document — approximately 300 words. Stage 2: T5 abstractive summarization reads those 300 words and generates a new 150-200 word summary. Two stages are necessary because T5 has a 512-token input limit and cannot process a full 50-page document directly. TF-IDF intelligently reduces the content while preserving the most important information.

Q15: What is UUID and why is it used for file naming?

A: UUID (Universally Unique Identifier) is a 128-bit random number expressed as a string like f47ac10b-58cc-4372-a567-0e02b2c3d479. `uuid.uuid4()` generates a completely random UUID. Each uploaded PDF is saved with this UUID as its filename. This prevents collisions — if two users upload files both named "policy.pdf", they get separate IDs and separate saved files on disk.

Q16: What is the role of pipeline.py?

A: `pipeline.py` is the orchestrator of the entire AI workflow. The `PolicyPipeline` class loads all models once in `__init__`. The `process()` method calls all other modules in correct sequence: extract text → preprocess → TF-IDF → T5 summary → key points → keywords → NER → simplification → return results. `main.py` only needs to call `pipeline.process(path)` — a single clean interface hiding all complexity.

Q17: How does React communicate with the Python backend?

A: React uses the browser's built-in `fetch()` function to make HTTP requests. When the user clicks "Analyse", `fetch` sends a POST request to `http://localhost:8000/summarize-policy?document_id=...` and awaits the response. FastAPI processes the request, runs the full AI pipeline, and returns JSON. React receives the JSON and re-renders the results section with all components.

Q18: What is the model used for citizen explanation?

A: `google/flan-t5-base` is used for citizen explanation. Flan-T5 is an instruction-tuned version of T5 — it was trained on 1,800+ NLP tasks with instruction-response pairs. This allows it to follow commands. The prompt says "Explain this policy in simple words for ordinary citizens. Use short sentences. Avoid jargon." The model adapts its output vocabulary and sentence structure based on this instruction.

Q19: What dataset does your project use?

A: This project does not use a fixed training dataset — it uses pre-trained models. T5/Flan-T5 was pre-trained by Google on the C4 corpus (750GB of web text). `spaCy en_core_web_sm` was trained on OntoNotes 5.0 and Common Crawl. NLTK WordNet is a lexical database of English. The "input data" for this project is any government policy PDF the user uploads — the system is designed to work on any English-language policy document.

Q20: What are the limitations of your project?

A: First: scanned/image PDFs cannot be processed — only digital PDFs with text layers. Second: processing takes 45–90 seconds on CPU which may feel slow. Third: T5 was not specifically fine-tuned on Indian government policy documents so some technical terms may not be perfectly handled. Fourth: very short documents (less than 1 page) produce minimal summaries. Fifth: the system works only for English-language documents — Hindi PDFs are not supported.

Quick Reference Cheat Sheet

Term	One-Line Definition
FastAPI	Python web framework — creates REST API endpoints, faster than Flask
NLTK	Python NLP library — tokenization, stopwords, lemmatization
spaCy	Production NLP — Named Entity Recognition using CNN model
TF-IDF	Statistical algorithm — scores sentences by word importance
T5	Google Transformer model — generates abstractive summaries
Flan-T5	Instruction-tuned T5 — follows commands like "explain simply"
Tokenization	Splitting text into sentences and words for processing
Stopwords	179 common English words removed before analysis
Lemmatization	Reduces words to root form: "policies" → "policy"
NER	Identifies named entities: organisations, dates, money, laws
Extractive	Selects existing sentences from document (TF-IDF method)
Abstractive	Generates new sentences summarising content (T5 method)
SQLite	File-based database, stored as policy.db, no server needed
UUID	Random unique ID for each file, prevents name collisions
REST API	Standard HTTP interface: GET/POST requests return JSON data
device=-1	HuggingFace parameter meaning: use CPU, not GPU
PyPDF2	Reads digital PDF files and extracts text layer
IOB tagging	NER labelling: B=beginning, I=inside, O=outside entity
Pipeline	Orchestrator class that calls all AI modules in sequence